

Rapport de gestion de projet — MVP chatbot RAG Puls-Events

Auteur : Morgan Le Gall - Formation Data Engineer, OpenClassrooms - juin 2026

Introduction

Puls-Events exploite une plateforme web de découverte d'événements culturels en temps réel, agrégeant des sources ouvertes (Open Agenda en tête) et personnalisant les résultats selon le lieu, la période et les thématiques de chaque utilisateur.

Un premier **Proof of Concept** (POC) a démontré la faisabilité d'un moteur de recherche sémantique : un chatbot de recommandation reposant sur une architecture **RAG** (Retrieval-Augmented Generation), couplant une base vectorielle et des modèles NLP pour répondre en langage naturel à des questions du type *« Quels concerts à Nantes ce mois-ci ? »*. Validé par les équipes produit et marketing, ce POC doit maintenant franchir une marche : devenir un **MVP** (Minimum Viable Product) capable de tenir la charge de plusieurs utilisateurs et de tourner en production.

Ce rapport constitue l'**étude de design** de ce MVP. Il synthétise les besoins, arrête une architecture cloud, propose un backlog priorisé et un plan de projet, et chiffre le coût de construction (build) et d'exploitation (OPEX).

Contexte et objectif du projet

Du POC au MVP

Le POC (projet P11) a livré une chaîne RAG fonctionnelle mais volontairement réduite, pour prouver la valeur sans sur-investir :

Dimension	POC livré	Caractère
Données	Open Agenda (Opendatasoft), Pays de la Loire , événements < 1 an	Périmètre restreint
Embeddings	mistral-embed (1024 dim)	Conservé
LLM	mistral-small-latest	Conservé
Vector store	FAISS local (IndexFlatL2)	Mono-machine, non partagé
Retrieval	LangChain, hybride BM25 + dense, MMR	Conservé et réutilisé
Interface	CLI + Streamlit	Démo locale, mono-utilisateur
Évaluation	20 paires Q/R annotées (hit_rate, cosine, LLM-as-judge)	Socle d'évaluation réutilisable

Le POC a été conçu **stateless** (sans mémoire), sans contexte géographique fin, sans recherche web, sans monitoring et sans hébergement cloud - choix assumés pour un POC. Ce sont précisément ces manques que le MVP doit combler.

Objectif du MVP

Transformer cette preuve de concept en un produit **multi-utilisateurs, scalable et observable**, déployé sur une infrastructure cloud, en ajoutant les quatre capacités attendues par l'équipe :

Mémoire conversationnelle - retenir et exploiter l'historique d'un utilisateur pour des interactions personnalisées.

Contexte géographique - adapter les réponses à la localisation de l'utilisateur.

Recherche web temps réel - compléter la base interne par une recherche en ligne (piste : smolagents de Hugging Face).

Monitoring de performance - mesurer l'efficacité du système et la satisfaction des utilisateurs.

Enjeu métier

Le projet sert la stratégie d'innovation de Puls-Events : se différencier sur un marché de l'événementiel où la personnalisation par l'IA devient un standard. Le MVP doit prouver que la valeur du POC tient à l'échelle, à un coût d'exploitation maîtrisé, et avec une qualité de réponse mesurable.

Analyse et synthèse des besoins

Besoins formulés par l'équipe

Les besoins exprimés par le responsable technique se déclinent en quatre capacités fonctionnelles, plus des exigences transverses implicites (scalabilité, coût, déploiement).

#	Besoin	Traduction technique	Critère de succès
B1	Mémoire conversationnelle	Persistance de l'historique + résumé des préférences ; injection du contexte de session dans le prompt RAG	Le chatbot tient compte des échanges précédents dans une même session et d'une session à l'autre
B2	Contexte géographique	Géolocalisation utilisateur + requêtes spatiales (rayon, distance, tri) sur les événements	« Que faire près de chez moi ce week-end ? » renvoie des événements triés par proximité réelle
B3	Recherche web temps réel	Agent capable de déclencher une recherche web quand la base interne ne suffit pas	Une question sur un événement très récent / hors base obtient une réponse sourcée du web
B4	Monitoring performance & satisfaction	Traçage des requêtes (latence, coût, qualité retrieval) + collecte de feedback utilisateur	Tableau de bord temps réel ; taux de satisfaction et latence p95 suivis
B5	Scalabilité (implicite)	Vector store et API capables de monter en charge ; ingestion automatisée	Supporte une montée de charge sans réécriture
B6	Maîtrise des coûts (implicite)	Architecture au coût marginal faible aux heures creuses	OPEX prévisible et optimisable

Synthèse du contexte, des objectifs métier et des contraintes techniques

Objectifs métier

- Personnaliser l'expérience de découverte culturelle (recommandations pertinentes, contextualisées).
- Étendre la couverture au-delà du périmètre POC (montée en charge données et trafic).
- Démontrer un produit déployable, observable et économiquement soutenable.

Contraintes techniques

- **Continuité avec le POC** : conserver les briques validées (Mistral, LangChain, retriever hybride, Docker) pour limiter le risque et le temps de réécriture.
- **Souveraineté / RGPD** : données et traitements de préférence hébergés en UE ; Mistral (LLM européen) cohérent avec cette contrainte.
- **Reproductibilité** : tout conteneurisé (Docker), infrastructure reproductible.
- **Coût** : un MVP doit minimiser l'OPEX aux faibles charges (privilégier le *scale-to-zero* / le serverless).
- **Données** : source Open Agenda via API Opendatasoft (accès ouvert), fraîcheur à maintenir par ingestion planifiée.

Utilisateurs cibles et cas d'usage

Personae

Persona	Profil	Besoin principal
Le curieux local	Habitant cherchant quoi faire près de chez lui	Recommandations géolocalisées, immédiates
Le planificateur	Prépare une sortie à une date / dans une ville données	Filtrage précis (lieu, période, thématique)
Le visiteur de passage	Touriste / déplacement professionnel	Découverte rapide d'événements à proximité, y compris très récents

Cas d'usage représentatifs

- **UC1 - Recherche géolocalisée** : *« Quels concerts ce soir à moins de 10 km ? »* → géolocalisation (B2) + retrieval + génération.
- **UC2 - Conversation à mémoire** : *« Et le week-end prochain ? »* après une première question → la session se souvient du lieu et de la thématique (B1).
- **UC3 - Événement hors base / très récent** : *« Y a-t-il une date ajoutée pour le festival X ? »* → l'agent bascule sur la recherche web (B3).
- **UC4 - Personnalisation inter-sessions** : un utilisateur revient ; le système se souvient qu'il aime le jazz et privilégie ce type d'événements (B1, mémoire long terme).
- **UC5 - Pilotage qualité** (interne) : l'équipe produit suit la latence, le coût et la satisfaction pour prioriser les améliorations (B4).

Plan de projet

Basé sur le chiffrage du backlog (≈ 60 j·h pour le MVP Must + Should), un développeur à temps plein. Échéancier en semaines relatives (S1 = démarrage du développement, après validation de cette étude de design).

Phasage et échéancier

Phase	Semaines	Fonctionnalités	Objectif
P0 - Étude de design	(fait)	Ce rapport	Vision validée, archi et stack arrêtées
P1 - Socle & infra	S1-S4	F1, F2, F3, F4, F5	RAG du POC migré sur pgvector, infra Scaleway provisionnée (IaC), API + CI/CD en place
P2 - Mémoire & géo	S5-S7	F6, F7, F8, F9	Conversations à mémoire + recherche géolocalisée fonctionnelles
P3 - Agent & web	S8-S9	F10, F11	Agent smolagents orchestrant RAG + recherche web temps réel
P4 - Monitoring & qualité	S10-S11	F12, F13, F14, F15	Observabilité complète (infra + RAG + satisfaction), éval non-régression en CI
P5 - Stabilisation & prod	S12	Durcissement, doc, bascule	MVP déployé en production

Jalons

Jalon	Échéance	Critère de franchissement	Livrable associé
J0 Design validé	Fin P0	Rapport approuvé par le resp. technique	Ce rapport + backlog + estimation coûts
J1 Socle opérationnel	Fin S4	RAG migré pgvector, API en ligne, CI/CD vert	Environnement dev déployé, démo API
J2 Mémoire & géo	Fin S7	UC1 (géo) et UC2 (mémoire) démontrés	Démo conversationnelle géolocalisée
J3 Agent web	Fin S9	UC3 (recherche web) démontré	Démo agent multi-outils

J4 Monitoring	Fin S11	Tableaux Cockpit + Langfuse actifs, feedback collecté	Dashboards + rapport d'éval
J5 MVP en production	Fin S12	Service prod stable, éval de non-régression au vert	MVP déployé + documentation

Livrables du projet

- **Code** : dépôt Git (API FastAPI, agent, pipeline Kestra, IaC Terraform), conteneurisé.
- **Documentation** : README, schéma d'architecture, runbook de déploiement, modèle de menace (hérité du POC).
- **Observabilité** : dashboards Cockpit + Langfuse.
- **Qualité** : jeu d'évaluation enrichi + rapport de métriques.
- **Gestion de projet** : ce rapport, backlog Excel, estimation des coûts Excel.

Risques projet et mitigations

Risque	Probabilité	Impact	Mitigation
Perf pgvector < FAISS sur gros volume	Moyenne	Moyen	Benchmark dès P1 ; chemin Qdrant documenté
Coût LLM qui dérape à la montée en charge	Moyenne	Élevé	Garde-fous agent + cache sémantique dès le départ
Qualité des adresses Open Agenda (géo)	Élevée	Moyen	Fallback ville/centroïde, géocodage tolérant aux adresses incomplètes
Courbe d'apprentissage IaC / Scaleway	Moyenne	Faible	Modules Terraform officiels, environnement dev jetable
Faible taux de feedback utilisateur	Moyenne	Faible	UX d'incitation discrète, métriques implicites de secours

Méthodologie de développement et documentation

- **Méthode itérative** : livraison par jalons démontrables (cf. 6.2), priorisation MoSCoW, ajustement du périmètre à chaque fin de phase.
- **Gestion du code** : Git, une branche par fonctionnalité, revue par *pull request* avant fusion, messages de commit normalisés.
- **Qualité** : tests pytest (unitaires + tests qualité des données hérités du POC : fraîcheur < 1 an, périmètre géo), évaluation RAG rejouée en CI (non-régression), *linting*.

- **CI/CD** : GitHub Actions enchaîne lint → tests (bloquants) → build d'image → déploiement sur Serverless Containers.
- **Infrastructure as Code** : Terraform versionné → l'infra est documentée et reproductible par construction.
- **Documentation technique** : README, schéma d'architecture, ADR (journal des décisions d'architecture), runbook de déploiement, modèle de menace (hérité du POC), docstrings.
- **Suivi de projet** : backlog priorisé, jalons datés, revue de fin de phase, indicateurs de délais/coûts/qualité.

Macro backlog des fonctionnalités

Priorisation **MoSCoW**. Complexité en T-shirt size (S/M/L/XL) ; délai en jours-homme (j·h) pour un développeur. Chaque ligne intègre le risque principal et sa mitigation.

Légende

- **Priorité MoSCoW** : ***Must*** (indispensable au MVP) · ***Should*** (important, non bloquant) · ***Could*** (souhaitable si temps) · ***Won't*** (hors périmètre MVP, noté pour la suite).
- **Complexité** : S ≈ 1-2 j·h · M ≈ 3-5 j·h · L ≈ 6-10 j·h · XL ≈ >10 j·h.

Backlog priorisé

ID	Épopée	Fonctionnalité	Priorité	Complexité	Délai (j·h)	Risque principal → mitigation
F1	Socle	Reprise du pipeline RAG du POC (ingestion, retriever hybride, éval)	Must	M	3	Dette POC → couvert par les tests existants
F2	Socle	Migration vector store FAISS → pgvector	Must	M	4	Écart de perf vs FAISS → benchmark + index HNSW
F3	Infra	Provisioning Scaleway via Terraform	Must	L	6	Courbe d'apprentissage IaC → modules

		(PG, Redis, containers, storage)				Terraform officiels Scaleway
F4	Infra	API FastAPI (endpoints chat, streaming SSE)	Must	M	4	—
F5	Infra	CI/CD GitHub Actions (tests bloquants → build → deploy)	Must	M	3	Secrets/clés → gestion via secrets Scaleway + GitHub
F6	B1 Mémoire	Mémoire court terme (Redis, fenêtre de session)	Must	M	3	Cohérence cache/DB → TTL + invalidation simple
F7	B1 Mémoire	Mémoire long terme + profil de préférences (PG + résumé LLM)	Should	L	6	Coût des résumés LLM → résumé incrémental, déclenché par seuil
F8	B2 Géo	Géocodage des événements + activation PostGIS	Must	M	4	Adresses incomplètes Open Agenda → fallback ville/centroïde
F9	B2 Géo	Recherche géolocalisée (filtre rayon, tri par distance)	Must	M	4	Combinaison géo + vectoriel → filtre PostGIS avant top-k
F10	B3 Web	Agent smolagents (orchestra	Must	L	7	Boucles d'agent coûteuses → limite d'itération

		tion des outils)				s + garde-fous coût
F11	B3 Web	Outil de recherche web temps réel (Tavily)	Should	M	3	Qualité/coût des résultats web → déclenchement conditionnel (base interne d'abord)
F12	B4 Monitoring	Observabilité infra (Cockpit : latence, CPU, logs)	Must	S	2	- (natif Scaleway)
F13	B4 Monitoring	Observabilité RAG + coût tokens (Langfuse)	Must	M	4	Self-hosting Langfuse → image officielle + PG dédié
F14	B4 Monitoring	Feedback utilisateur (👍/👎) + tableau de satisfaction	Should	M	3	Faible taux de feedback → UX d'incitation discrète
F15	B4 Monitoring	Éval qualité jouée en CI (non-régression)	Should	S	2	Seuils mal calibrés → marges sur le jeu annoté
F16	Produit	Frontend web conversationnel soigné	Could	L	8	Périmètre extensible → MVP sur composant chat minimal
F17	Produit	Extension du périmètre géographique (national)	Could	M	4	Volume de données ×N → valider la scalabilité pgvector d'abord

F18	Produit	Notifications / alertes événements personnalisés	Won't (MVP)	L	—	Reporté post-MVP
F19	Produit	Support multilingue	Won't (MVP)	M	—	Reporté post-MVP

Synthèse de l'effort

Périmètre	Fonctionnalités	Charge cumulée
Must (MVP minimal livrable)	F1–F6, F8–F10, F12–F13	≈ 44 j·h
+ Should (MVP complet recommandé)	+ F7, F11, F14, F15	≈ 60 j·h
+ Could (si marge)	+ F16, F17	≈ 72 j·h

Le **MVP recommandé** (Must + Should) représente environ **60 jours-homme**, soit ~12 semaines pour un développeur, ou ~6 semaines à deux. Ce chiffrage alimente le plan de projet (jalons) et l'estimation des coûts de build.

Architecture technique détaillée

Vue d'ensemble

L'architecture conserve le cœur RAG validé au POC (Mistral + LangChain + retriever hybride) et l'industrialise sur **Scaleway** (cloud souverain UE). Le principe directeur est de **consolider** : une seule base PostgreSQL porte le relationnel, les vecteurs (**pgvector**), la géo (**PostGIS**) et la mémoire long terme, au lieu de quatre systèmes distincts.

La trajectoire part directement du POC RAG Puls-Events : on conserve son pipeline (ingestion batch + RAG runtime) et on le fait évoluer brique par brique, en fonction des quatre besoins. La figure 1 met en regard l'hérité et le nouveau ; la figure 2 détaille la cible de déploiement sur Scaleway.

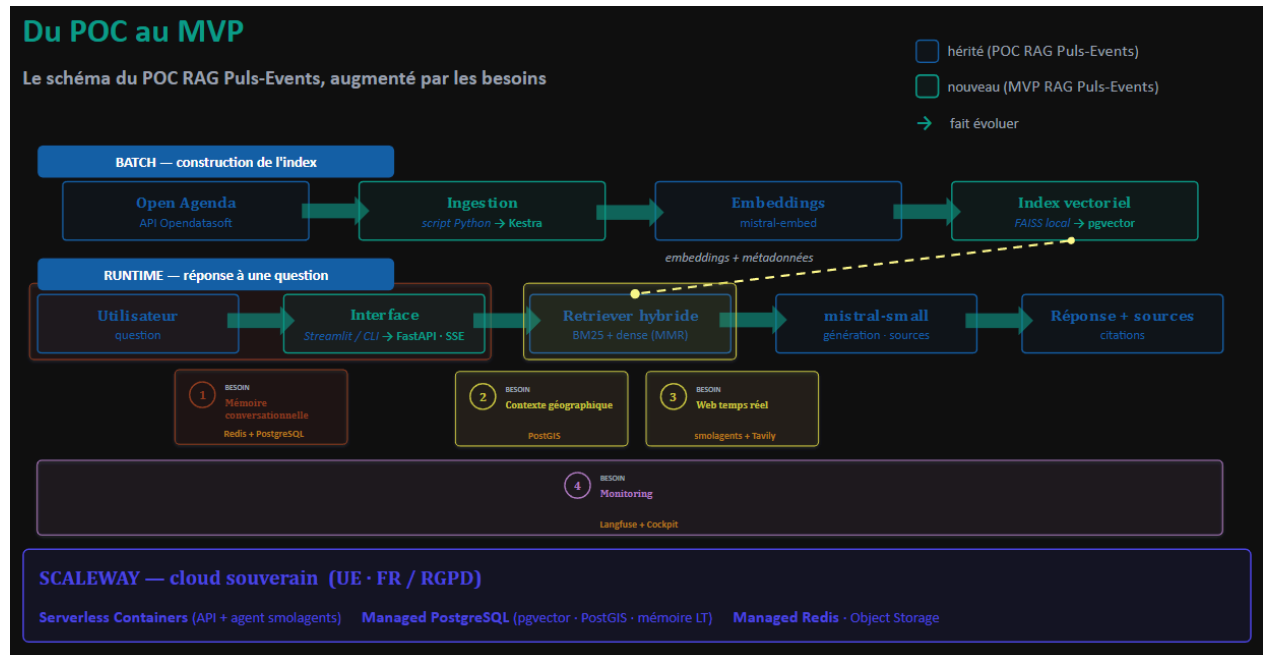


Figure 1 - Du POC P11 au MVP : le pipeline du POC, augmenté par les besoins. En bleu, le socle hérité (retriever hybride BM25+dense, Mistral, réponses sourcées) ; flèches « → » : évolutions d'infra (FAISS → pgvector, Streamlit → FastAPI, scripts → Kestra) ; les quatre besoins P13 greffés ; socle Scaleway en bas.

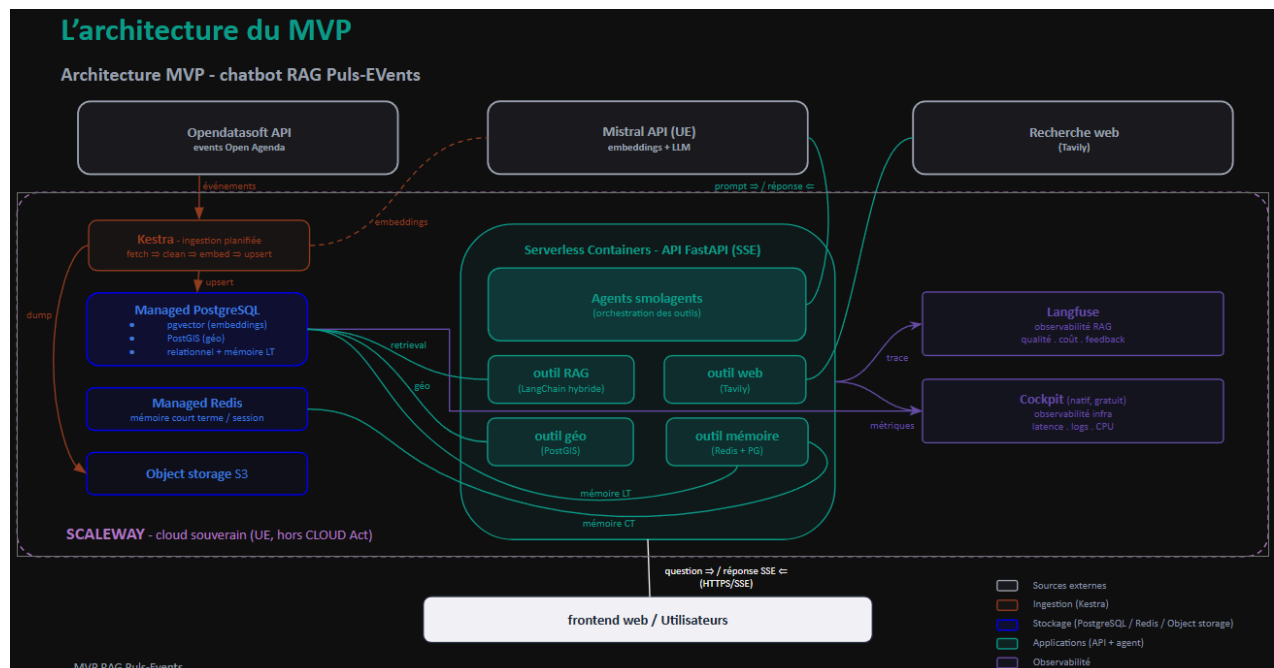


Figure 2 - Déploiement détaillé sur Scaleway (cloud souverain UE, hors CLOUD Act) : ingestion Kestra, base PostgreSQL consolidée, agent smolagents et double observabilité Langfuse + Cockpit.

Composants et services

Brique	Service Scaleway / techno	Rôle	Défi couvert
API backend	Serverless Containers (FastAPI, async, SSE)	Point d'entrée chat, scalable, pay-per-use	B5
Agent	smolagents (HF)	Orchestre les outils (RAG, géo, mémoire, web)	B3
RAG	LangChain (retriever hybride BM25+dense, MMR)	Réutilisé du POC P11	—
Base principale	Managed PostgreSQL + pgvector + PostGIS	Relationnel + vecteurs + géo + mémoire long terme	B2, B5
Cache / session	Managed Redis	Mémoire conversationnelle court terme	B1
Stockage objets	Object Storage (S3-compatible)	Dumps bruts Open Agenda écrits par Kestra (reconstructibilité de l'index sans réinterroger la source)	—
Ingestion	Kestra (container)	Pipeline planifié fetch→clean→embed→upsert	B5
LLM + embeddings	Mistral API (mistral-small, mistral-embed)	Génération + vectorisation (UE)	—
Recherche web	Tavily (ou DuckDuckGo)	Outil web temps réel de l'agent	B3
Observabilité RAG	Langfuse (container)	Traces, qualité retrieval, coût tokens, feedback 👍 / 👎	B4
Observabilité infra	Cockpit (natif Scaleway)	Logs, métriques, latence, CPU — gratuit, actif d'office	B4
Registre images	Container Registry (auto-provisionné)	Images Docker	—

Flux de données

Flux d'ingestion (asynchrone, planifié par Kestra)

1. Récupération paginée des événements Open Agenda via l'API Opendatasoft (filtres période/zone).
2. Nettoyage (HTML, doublons par uid), normalisation, géocodage des lieux.
3. Découpage en chunks → embeddings `mistral-embed` → *upsert* dans PostgreSQL (pgvector), métadonnées et coordonnées (PostGIS) dans la même base.
4. **Kestra** conserve un dump brut dans l'**Object Storage** (S3-compatible) avant ingestion dans PostgreSQL — restructibilité de l'index sans réinterroger Open Agenda.

Flux de requête (synchrone, temps réel)

1. L'utilisateur envoie une question (avec sa géolocalisation) à l'API FastAPI.
2. La mémoire court terme (Redis) et le profil long terme (PostgreSQL) sont chargés et injectés dans le contexte.
3. L'agent smolagents décide des outils : retrieval hybride (filtré géographiquement par PostGIS), et/ou recherche web si la base interne est insuffisante.
4. Mistral génère la réponse à partir des chunks récupérés ; les sources sont jointes.
5. La réponse est streamée (SSE) ; l'échange est persisté (mémoire) et tracé (Langfuse).

Choix du cloud provider — veille technique

Trois candidats évalués, sur les critères MVP : capacité à monter en charge, support pgvector/PostGIS, souveraineté/RGPD, coût, écosystème.

Critère	GCP Cloud Run	Scaleway (retenu)	AWS Fargate
Scale-to-zero (OPEX idle)	✓ vrai zéro, free tier 2M req/mois	⚠ Serverless Containers pay-per-use	✗ pas de scale-to-zero natif
pgvector + PostGIS	✓ Cloud SQL	✓ Managed PostgreSQL (>30 extensions)	✓ RDS
Redis managé	✓	✓ Managed Database for Redis	✓
Observabilité incluse	partielle	✓ Cockpit gratuit (Grafana/Loki/Mimir/Tempo)	✗ CloudWatch facturé
Souveraineté / RGPD	✗ CLOUD Act (entreprise US)	✓ juridiction UE garantie , siège Paris, DC UE	✗ CLOUD Act (même l'European Sovereign Cloud)
Transparence des coûts	bonne	✓ rates publiés, pas de frais cachés	⚠ frais cachés (IPv4, NAT, ALB → compute ≈ 68 % de la facture)
Coût à forte charge (>20M req/mois)	⚠ explose (par requête)	✓ forfait prévisible	moyen
Écosystème ML / employabilité	✓ très riche	⚠ plus modeste	✓ très riche

Décision : Scaleway. Justification :

- **Souveraineté** - donnée décisive ici. Les fournisseurs US (GCP, AWS) restent soumis au *CLOUD Act* même via leurs offres « souveraines » européennes ; seul un fournisseur de droit européen garantit que la donnée reste sous juridiction UE. Pour une plateforme française manipulant des données utilisateurs (préférences, localisation), c'est un atout de conformité RGPD et un argument commercial.
- **Cohérence de la stack** - Scaleway (FR) + Mistral (LLM FR) + données culturelles publiques françaises forment un ensemble cohérent et défendable.
- **Coût et transparence** - tarifs publiés sans frais cachés, observabilité (Cockpit) incluse gratuitement, et coût prévisible à la montée en charge (là où Cloud Run explose au-delà de ~20M requêtes/mois).
- **Couverture technique** - pgvector, PostGIS, Redis managé, Serverless Containers et Registry couvrent 100 % des besoins du MVP.

Compromis assumé : écosystème ML plus modeste que GCP/AWS et absence de vrai scale-to-zero. Le second est mineur pour un MVP au trafic modéré (instance Serverless

minimale peu coûteuse) ; **GCP est conservé comme plan B documenté** si un besoin ML managé avancé (AutoML, TPU) émergeait.

Stratégie de déploiement

- **Conteneurisation** : chaque service (API FastAPI, Kestra, Langfuse) packagé en image **Docker**, poussée vers le **Container Registry** Scaleway puis déployée sur Serverless Containers.
- **Infrastructure as Code : Terraform** (provider Scaleway officiel) provisionne l'ensemble — PostgreSQL, Redis, Serverless Containers, Object Storage, Cockpit — et déclare les images Docker issues du Container Registry comme source de déploiement. Infrastructure et runtime coévoluent dans le même dépôt Git.
- **CI/CD** : pipeline GitHub Actions (lint → tests `pytest` → build image → push registry → déploiement Serverless Containers). Les tests qualité hérités du POC (fraîcheur < 1 an, périmètre géo) restent bloquants.
- **Environnements** : dev (base DEV-S à ~11 €/mois) et prod (instance HA), isolés par projet Scaleway.

Stratégie de modularité

- **Séparation des responsabilités** : ingestion (Kestra) / indexation / API RAG / agent / observabilité sont des modules indépendants, déployables et scalables séparément.
- **Abstraction du LLM** : la couche Mistral est encapsulée derrière l'interface LangChain → un changement de modèle (Mistral ↔ autre) ne touche pas le reste du code.
- **Outils d'agent enfichables** : sous smolagents, chaque capacité (RAG, géo, mémoire, web) est un outil ajoutable/retirable sans refonte.
- **Réutilisation du POC** : le retriever hybride, le preprocessing et le jeu d'évaluation P11 sont repris tels quels — dette technique minimale.
- **Chemin de montée en charge documenté** : si le volume de vecteurs dépasse les capacités de pgvector, extraction vers **Qdrant** sans toucher à la logique métier (interface vector store isolée).

Stratégie de monitoring

Deux niveaux complémentaires couvrent le défi B4 (efficacité **et** satisfaction) :

Niveau	Outil	Mesures	Question métier
Infrastructure	Cockpit (Scaleway)	Latence API, CPU/mémoire, taux d'erreur, logs	Le système tient-il la charge ?
Qualité RAG	Langfuse	Traces requêtes, qualité retrieval (hit_rate), coût tokens, latence par étape	Les réponses sont-elles pertinentes et au bon coût ?
Satisfaction	Langfuse (feedback)	Notes 👍 / 👎 utilisateur, taux de réponses jugées utiles	Les utilisateurs sont-ils satisfaits ?
Métier	Tableau de bord Cockpit	Latence p95, taux de satisfaction, nb requêtes/jour	Faut-il prioriser des améliorations ?

Cockpit et Langfuse sont complémentaires : Cockpit surveille la couche infra (santé des conteneurs, latence réseau, CPU) ; Langfuse trace la couche RAG (qualité des réponses, coût tokens, feedback utilisateur). Un pic CPU dans Cockpit sans dégradation Langfuse signale un problème infra ; une baisse de hit_rate sans alerte Cockpit signale un problème de corpus ou de modèle.

Le jeu d'évaluation annoté du POC (20 paires Q/R, métriques hit_rate / cosine / LLM-as-judge) est rejoué en CI à chaque déploiement pour détecter toute régression de qualité avant la mise en production.

Optimisation de l'index vectoriel (latence & pertinence)

Le POC utilisait FAISS IndexFlatL2 : recherche **exacte**, parfaite sous ~100 k chunks mais coûteuse à l'échelle (comparaison à tous les vecteurs). Le MVP passe à un index **approximatif** (ANN) sur pgvector. Deux options, à arbitrer selon le volume :

Index	Construction	Mémoire	Rappel / latence	Paramètres clés
HNSW (recommandé MVP)	Plus lente	Plus élevée	Excellent rappel, latence basse	m (connexions/n œud), ef_construction (build), ef_search (requête)
IVFFlat	Rapide	Faible	Bon, sensible au réglage	lists ($\approx \sqrt{n}$ partitions), probes (requête)

Leviers de latence

- ef_search (HNSW) ou probes (IVFFlat) règlent directement le compromis **rappel** ↔ **latence** : on les calibre sur le jeu d'évaluation (cible : hit_rate maintenu, latence p95 minimale).
- **Filtrage géographique d'abord** : PostGIS réduit l'espace de recherche (événements dans le rayon) *avant* la recherche vectorielle → moins de vecteurs à parcourir, latence moindre.
- **Cache d'embeddings** (hérité du POC) : aucune requête n'est ré-embeddée inutilement.

Leviers de pertinence

- **Retriever hybride BM25 + dense** (hérité du POC) : le lexical rattrape les noms propres (Nantes, Hellfest) que le sémantique peut rater ; fusion par Reciprocal Rank Fusion.
- **MMR** pour diversifier le top-k (éviter 4 chunks du même festival).
- **Re-ranking cross-encoder** (bge-reranker-v2-m3, self-hosted) : implémenté et benché (reco v1 du POC P11), puis **désactivé par défaut** — la donnée le dit. Sur le jeu d'évaluation, en comparaison **loyale à k constant** (recall@6, même pool de candidats), le cross-encoder **dégrade** le recall : **0.86** → **0.64**. Diagnostic par question : il sacrifie la précision lexicale de BM25 (ex. requête *« événements gratuits »* → le doc « entrée gratuite » passe du rang 0 au rang 7), au profit d'événements sémantiquement proches mais erronés — le corpus regorge de quasi-doublons. Coût annexe : **~69 s/requête en CPU**. Conclusion *pilotée par la donnée* : le moteur hybride BM25+dense seul est meilleur ici ; la feature reste branchée (toggle use_reranker, ré-activable par USE_RERANKER=true) pour un corpus moins redondant ou une exécution GPU. La démarche — mesurer avant d'affirmer un gain — est elle-même le livrable.

- **Tokenisation française du BM25** : le moteur lexical utilisait le tokeniseur par défaut (découpage sur les espaces, sans normalisation). Une tokenisation FR a été implémentée et benchée (dé-accentuation, retrait des mots-outils, racinisation Snowball) pour rapprocher les formes (*« gratuits »* → *« gratuit »*). En comparaison **loyale à k constant** (recall@6), le **gain est nul** : **0.86** → **0.86** (0 question améliorée, 0 dégradée sur 14). Diagnostic : sur ce jeu, l'hybride BM25+dense **égale déjà le dense seul** — le BM25 n'apporte aucun document au top-6, donc améliorer sa tokenisation ne peut rien changer. La feature reste branchée (toggle use_fr_tokenizer, ré-activable par USE_FR_TOKENIZER=true) pour une réévaluation sur une métrique plus fine (MRR, recall@3) ou un corpus moins dominé par le dense. Second résultat négatif assumé : vérifier qu'un composant **contribue** à la métrique avant de l'optimiser.

Mesure continue : hit_rate@k (qualité retrieval) et latence par étape sont tracés dans Langfuse → le réglage de l'index est piloté par la donnée, pas à l'aveugle.

Estimation des coûts (build & OPEX)

Sera exporté en Excel (livrable annexe). Tarifs vérifiés à la source en juin 2026 (Mistral, Scaleway). Mistral facture en USD ; conversion indicative au taux $\approx 1 \text{ USD} = 0,93 \text{ €}$ (à réactualiser).

Coût de build (développement initial, one-shot)

Le poste dominant est la main-d'œuvre. Hypothèse : coût chargé d'un développeur $\approx 500 \text{ €/jour}$.

Poste	Charge (j·h)	Coût (≈)
Étude de design (ce rapport)	10	5 000 €
Développement MVP (Must + Should)	60	30 000 €
Tests, intégration, déploiement, durcissement	10	5 000 €
Total build	80 j·h	≈ 40 000 €

Coûts cloud pendant le build : marginaux (environnement dev, base DEV-S $\sim 11 \text{ €/mois}$, indexation initiale Mistral $\sim 0,30 \text{ €}$). Négligeables face à la main-d'œuvre.

Modèle de coût d'une requête (variable clé de l'OPEX)

Une requête RAG consomme :

- **Input** $\approx 3\,500$ tokens (prompt système + historique + 4 chunks de contexte + question).
- **Output** ≈ 400 tokens.
- Coût Mistral : $(3\,500 \times \$0,10 + 400 \times \$0,30) / 1 \text{ M} \approx \$0,00047 \approx 0,0004 \text{ €}$.
- Surcoût agent (smolagents peut enchaîner 2-3 appels LLM par requête) : on retient une moyenne de $\approx 0,001 \text{ €/requête}$.

L'ingestion (embeddings) est négligeable : $\sim 5\,000$ événements $\approx 0,30 \text{ €}$ par réindexation complète, avec cache d'embeddings hérité du POC.

OPEX par palier de charge (coût mensuel)

Poste	Palier 1 — Lancement (1 k users, ~10 k req/mois)	Palier 2 — Croissance (10 k users, ~150 k req/mois)	Palier 3 — Montée en charge (100 k users, ~2 M req/mois)
Mistral API (LLM + embed)	≈ 10 €	≈ 150 €	≈ 2 000 €
Serverless Containers (API)	0 € (free tier)	≈ 20 €	≈ 150 €
Managed PostgreSQL (pgvector+PostGIS)	≈ 80 € (PRO2-XXS)	≈ 270 € (PRO2-XS + HA)	≈ 500 €
Managed Redis	≈ 35 €	≈ 70 €	≈ 150 €
Object Storage	≈ 5 €	≈ 10 €	≈ 30 €
Hébergement Kestra + Langfuse	≈ 30 €	≈ 40 €	≈ 100 €
Cockpit (observabilité)	0 € (inclus)	0 €	0 €
Total OPEX / mois	≈ 160 €	≈ 560 €	≈ 2 930 €

Lecture : aux faibles charges, l'OPEX est dominé par l'**infrastructure fixe** (base, Redis) ; à grande échelle, il bascule vers le **coût d'usage Mistral** (~68 % au palier 3). C'est donc le coût LLM qu'il faut optimiser en priorité quand le trafic croît.

Propositions d'optimisation budgétaire

Levier	Effet	Quand l'activer
Cache sémantique des réponses fréquentes	Évite l'appel LLM sur les questions récurrentes (peut couper 20-40 % des appels)	Dès le palier 2
Réduction du contexte (k plus petit, compression des chunks)	Moins de tokens input → coût Mistral réduit	Palier 2-3
Garde-fous agent (limite d'itérations, déclenchement web conditionnel)	Plafonne le coût des requêtes agentiques	Dès le lancement
Réindexation incrémentale (vs complète)	Réduit les coûts d'embedding	Dès le lancement
Scale-to-zero des services non critiques (Kestra ne tourne qu'à l'ingestion)	Supprime le coût des composants inactifs	Dès le lancement
Cockpit (observabilité gratuite) vs solution facturée	Économie directe d'OPEX	Acquis (choix Scaleway)
Modèle adapté à la tâche (mistral-small par défaut, escalade ponctuelle)	Évite de payer un gros modèle là où le petit suffit	Continu

Avec ces leviers, l'OPEX au palier 3 peut être ramené de ~2 930 € à un ordre de **2 000-2 200 €/mois** à charge égale, principalement via le cache sémantique et la réduction du contexte.

Bilan

Du POC au MVP — étapes clés

POC (P11) : faisabilité prouvée - chaîne RAG Mistral + LangChain + FAISS sur Open Agenda (Pays de la Loire, < 1 an), évaluée sur 20 paires Q/R.

Étude de design (ce projet) : cadrage des besoins, choix d'architecture cloud souveraine, backlog priorisé, chiffrage build/OPEX, plan de projet.

MVP cible : industrialisation sur Scaleway, ajout des 4 capacités manquantes (mémoire, géo, web, monitoring), passage mono-utilisateur → multi-utilisateurs scalable et observable.

Justification des choix techniques et méthodologiques

- **Continuité avec le POC** (Mistral, LangChain, retriever hybride, Docker) : réduit le risque et le temps de réécriture ; capitalise sur l'évaluation déjà construite.
- **FastAPI** (async, SSE) : le POC exposait un CLI et une UI Streamlit mono-utilisateur. FastAPI fournit des endpoints HTTP stateless, le streaming SSE des réponses token-par-token, et une intégration native avec smolagents — sans dépendance frontend spécifique, compatible avec une montée à plusieurs utilisateurs simultanés.
- **Consolidation sur PostgreSQL** (pgvector + PostGIS + relationnel + mémoire) : une seule base au lieu de quatre systèmes → architecture plus simple, OPEX moindre, requêtes géo + vectorielles combinées nativement.
- **Scaleway** : souveraineté FR/RGPD (hors CLOUD Act), cohérence avec Mistral, observabilité incluse gratuitement, coût transparent et prévisible.
- **Agent smolagents** : la recherche web (B3) n'est pas un simple appel d'API en dur — le système doit *décider* quand la base interne est insuffisante, reformuler la requête et recombinaison les sources. Cette logique relève d'un agent (boucle de raisonnement type ReAct). smolagents (Hugging Face) est retenu pour trois raisons : il est LLM-agnostique (réutilise Mistral via LiteLLM, sans rupture RGPD), ses *CodeAgents* expriment les enchaînements d'outils en Python exécutable — plus compacts et fiables qu'un agent à appels JSON — et son cœur minimaliste limite le coût d'intégration et les dépendances transitives. Chaque capacité (RAG, géo, mémoire, web) devient un outil enfichable, ajoutable sans refonte. L'arbitrage assumé : LangChain, déjà présent, propose aussi des agents, mais on cantonne son usage au *retrieval* RAG et on délègue la branche agentique à smolagents plutôt que d'empiler LangGraph ; le surcoût (un second framework d'orchestration) est borné par des garde-fous d'itérations et de coût (F10).
- **Méthode** : priorisation MoSCoW, livraison par jalons démontrables, qualité garantie par l'évaluation rejouée en CI.

Défis rencontrés et solutions apportées

Défi	Solution retenue
Rendre le RAG stateful (mémoire)	Architecture mémoire court terme (Redis) + long terme (PostgreSQL + résumé LLM)
Pertinence géographique	PostGIS dans la même base que les vecteurs → filtrage spatial avant le top-k
Compléter une base interne forcément incomplète	Agent avec outil de recherche web déclenché conditionnellement
Mesurer efficacité et satisfaction	Double observabilité Cockpit (infra) + Langfuse (RAG, coût, feedback)
Maîtriser le coût LLM à l'échelle	Cache sémantique, réduction de contexte, garde-fous agent
Souveraineté des données utilisateurs	Hébergement intégral sur cloud de droit européen

Conclusion

Le POC a prouvé qu'un chatbot RAG apporte une vraie valeur de découverte culturelle. Cette étude de design montre comment le transformer en un MVP **scalable, observable, économiquement soutenable et conforme au RGPD**, sans renier les briques déjà validées. L'architecture proposée — cœur RAG du POC industrialisé sur Scaleway, données et IA souveraines, observabilité native — couvre les quatre défis posés par l'équipe et trace un chemin de montée en charge documenté. Le plan de projet (≈ 12 semaines, ~40 000 € de build, OPEX maîtrisé de 160 € à ~2 900 €/mois selon la charge) donne au commanditaire une trajectoire claire et chiffrée pour passer à l'exécution.

Annexes

Portfolio professionnel

Portfolio GitHub regroupant les projets du parcours Data Engineer (description, outils/technologies, résultats avec démonstrations et benchmarks, compétences démontrées et valeur ajoutée).

- Portfolio GitHub : <https://github.com/MoyLiG/data-engineer-portfolio.git>
- POC RAG (P11) : système de recherche sémantique sur événements culturels.
- Projets data/infra connexes (P10, P12) : orchestration Kestra, infrastructure data.

Références (veille technique)

- Mistral — tarifs API : <https://mistral.ai/pricing>
- Scaleway — tarifs : <https://www.scaleway.com/en/pricing/>
- Scaleway — Managed PostgreSQL (pgvector, PostGIS) : <https://www.scaleway.com/en/managed-postgresql-mysql/>
- Scaleway — Serverless Containers : <https://www.scaleway.com/en/pricing/containers/>
- Scaleway — Managed Database for Redis : <https://www.scaleway.com/en/managed-database-for-redis/>
- Scaleway — Cockpit (observabilité) : <https://www.scaleway.com/en/cockpit/>
- AWS European Sovereign Cloud & CLOUD Act (InfoQ, jan. 2026) : <https://www.infoq.com/news/2026/01/aws-european-sovereign-cloud/>
- Souveraineté numérique UE — guide 2026 : <https://gartsolutions.com/digital-sovereignty-of-europe/>
- Hugging Face — smolagents : <https://github.com/huggingface/smolagents>
- Langfuse — observabilité LLM : <https://langfuse.com>